Quickstart

Introduction

In this guide we'll setup a simple 2D pathfinding **grid**, add obstacles, and then generate paths for a pathfinding **agent**.

> **Finished scene**
>
> For the finished version, check out: `JPS4Pathfinding2D/Samples/Scenes/minimal_example.unity`

Creating the 2D Grid

To get started, we'll create an empty scene and add our **grid**.

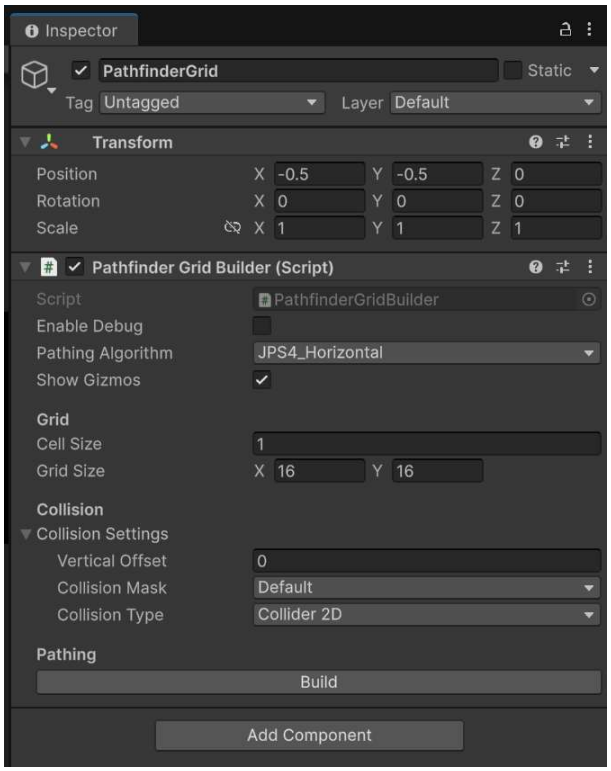
1. Create an empty GameObject and add the `PathfinderPathfinderGridBuilder` component.

   > *You can have multiple grids in a scene. These primary store collision (obstacles) for use across multiple pathfinding agents.*

2. Ensure Gizmos are turned on in the editor scene view and the ☑ `Show Gizmos` flag is checked on the `PathfinderGridBuilder` component

3. Now choose a grid size for the area of the grid (we'll use 16x16 for now). Lastly set the world size of a cell *(we'll use unit = 1 for now)*. Press the "Build" button on the component. You should see the the grid now in the scene view & can adjust the camera to fit.

Grid Settings



Here you can configure the pathfinding settings that will be used with this grid.

🄴 **Pathing Algorithm**: Which algorithm will be used when generating paths with this grid.

☑ **Show Gizmos:** Show gizmos for the grid cells & obstacles?

⊞ **Cell Size:** the world size of a grid cell.

📏 **Grid Size:** the number of cells (width x length) for each side of the grid.

⌖ **Collision Settings:** Configure how obstacles are detected. Collision is generate by calling `Build` on the **grid**.

Building Collision & Obstacles

""Building" the grid here just means storing the current traversable state of each cell in the grid for pathfinding.

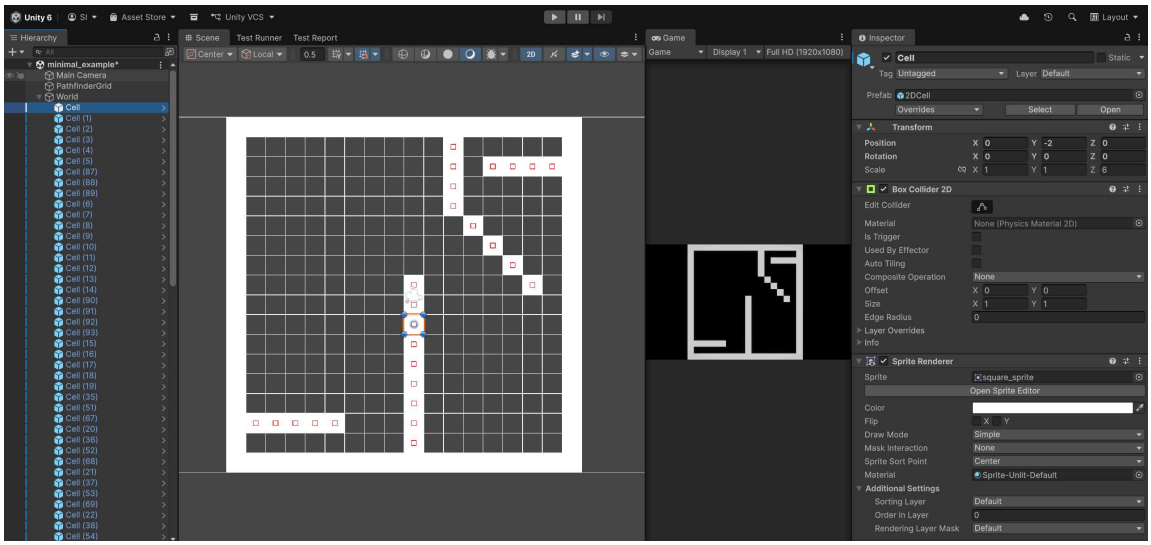
Caching

The strength of dynamic A* based algorithms is that nothing needs to be cached, so we're just storing the collision here for use across multiple jobs. This only needs to be done **once** if obstacles are static. If obstacles move/change, we just need to "re-build" the collision for the grid before scheduling any more pathing jobs.

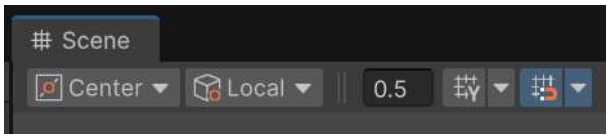
In regard to agent movement, you can simply recalculate paths on moving agents when the grid is re-built.

The **grid** can be built manually by providing a `NativeArray<bool> GridTraversable` to the `PathfinderGridBuilder`. For convenience there is built-in collision detection using the **Collision Settings** on the `PathfinderGridBuilder` component.

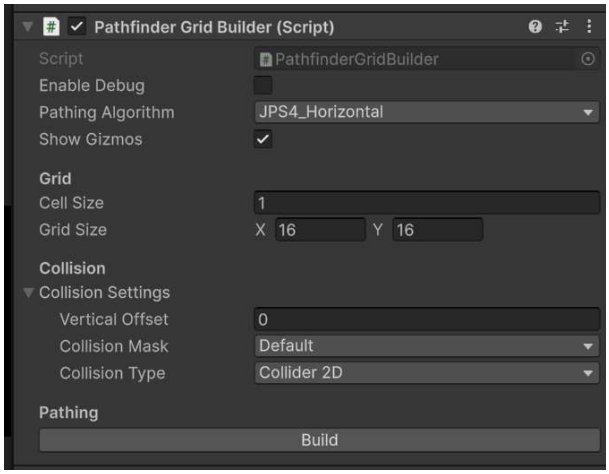
1. Select the `PathfinderGridBuilder` Object and set the collision settings:
    - **Collision Type:** Select "Collider 2D" to use 2D raycasting (since we will be using BoxCollider2D for cell collision masks).
    - **Collision Mask** Set the flags to the "Default" layer only - this is the layer we're raycasting against for collision.
2. Now we just need to make the grid cell objects. Under an empty GameObject "World", add Cell GameObject `SpriteRenderers` with `BoxCollider2D` components.



3. Move the obstacle objects to align with the grid. You can turn on snapping to make this easier. *(If you're not seeing the grid just click "Build" again on the `PathfinderGridBuilder` GameObject)*



4. Press "Build" button on the `PathfinderGridBuilder` once all obstacles are setup: you should see red squares for the obstacles, and the cell objects should be aligned with the grid. *If not, move the GridBuilder transform to be centered and double check your object layers & collision settings.*

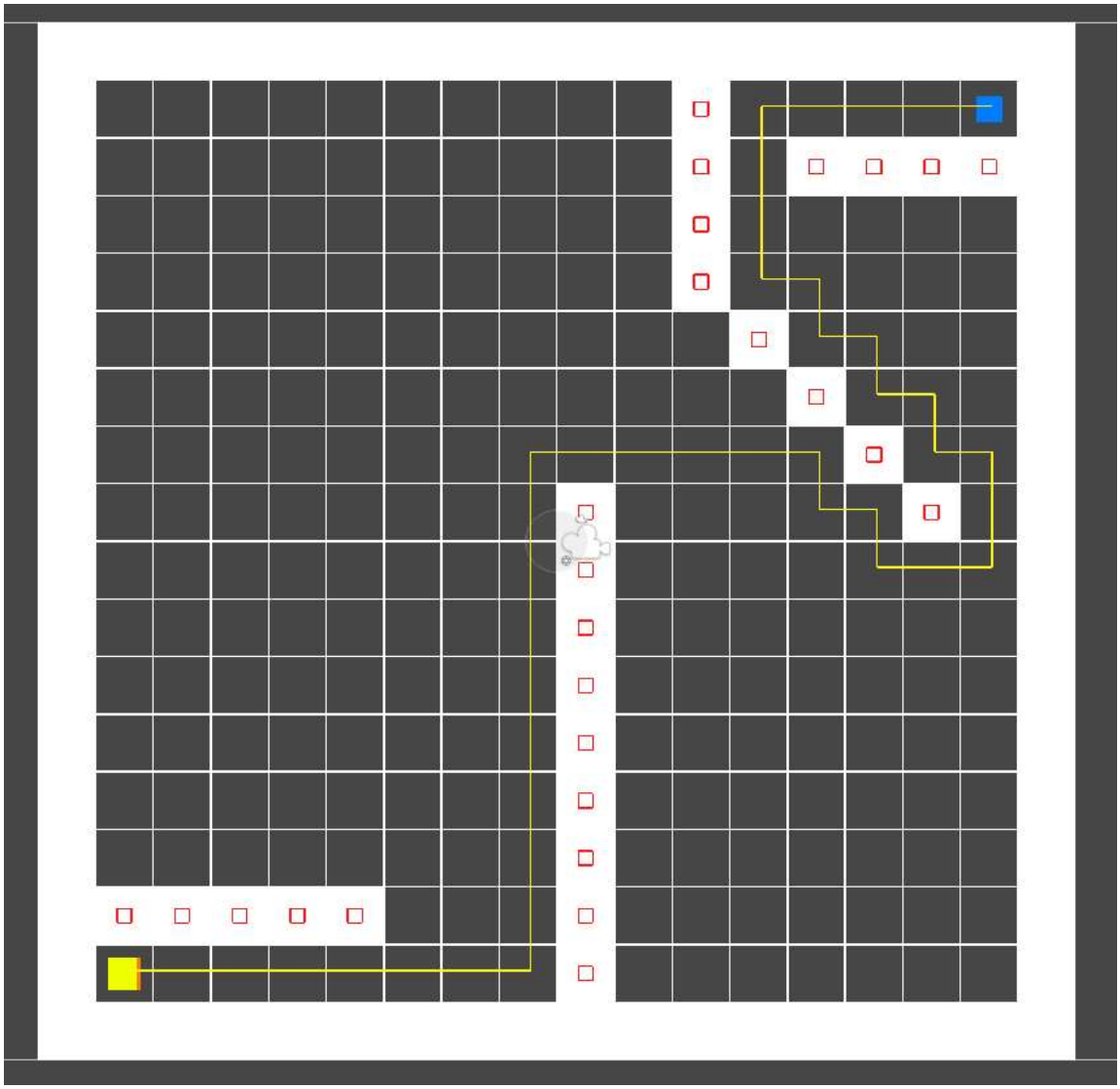


Creating a Pathing Agent

Now let's make a **pawn** or **agent** to generate paths on the grid.

> **Custom Agents**
>
> The existing script is mainly meant to be example to reference. For more customized behavior, create your own agents and just call the same `PathfinderGridBuilder` methods to generate paths. There is no dependency from the builder to the existing agent scripts.

1. Let's add two new sprite renderer objects, one as the "Pawn" and one as the target. Ensure both are set to a Layer other than "Default" (ex. Ignore Raycast) so they aren't detected as obstacles. *(Remember you can pass your own collision arrays to the builder if you're already using layers for something else and can't compromise on your agent object)*
2. Add a `PathfinderAI` component to the pawn and assign the grid and target references.
3. Now we can make the path! Press "Generate Path" on the `PathfinderAI` component.



Moving Agents

The last aspect is moving the pathfinding agents. This package includes helper movement utilities: let's give it a shot!

1. Add a `DiscreteMovementComponent` to the pawn GameObject that is serving as our agent.
2. We don't need rotation for now, so let's just set the "Move Anim" variables:
    - **Dur:** [0.1] the duration of each movement step.
    - **Delay** [0] the time in between each movement
    - **Curve** [Linear] the interpolation curve for each movement step.

3. Enter PlayMode in the Editor to test the movement. First "Generate Path" on the `PathfinderAI` component, the press "Move Along Path"

That's it!

We've now covered the basics of setting up a 4-connected grid and pathfinding agent. Following are some additional helpful sections for quick reference, but be sure to read the rest of the docs for more detail.

Choosing a Pathing Algorithm

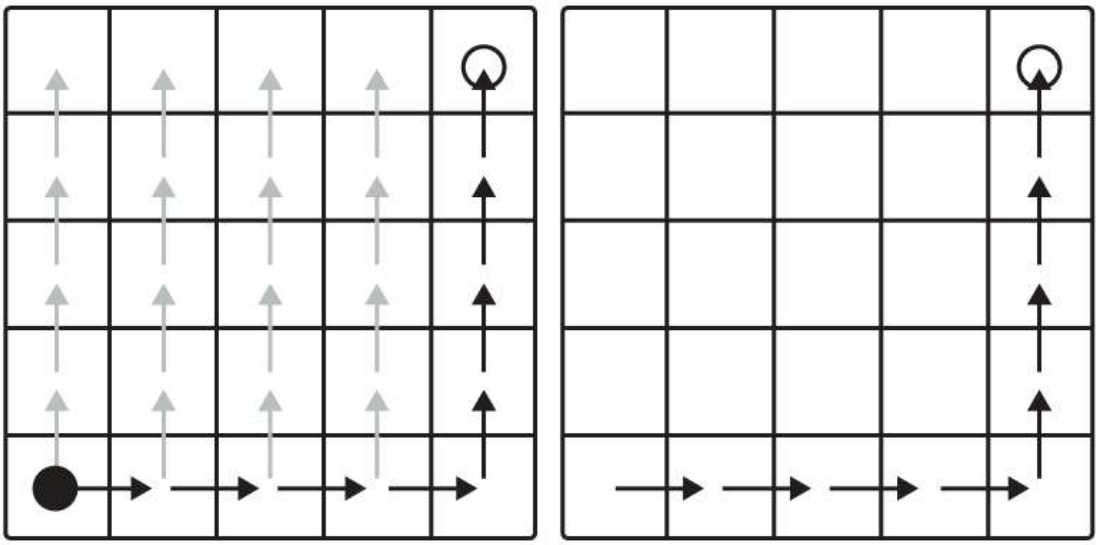
The package includes various pathfinding algorithms that will change the shape of the path (when there is more than one optimal path available). If you're unsure of which to use, try `JPS4_Horizontal` first for the best performance, then `AStar4` after.

Horizontal or Vertical JPS4?

These two modes specify the "canonical ordering" of the jump points used in the **JPS4 Algorithm.** A canonical ordering is a total ordering over paths which can eliminate many of the redundant paths. This is what drives the main performance advantage of JPS4 - it will also result in very different paths (when there is 2+ optimal paths).

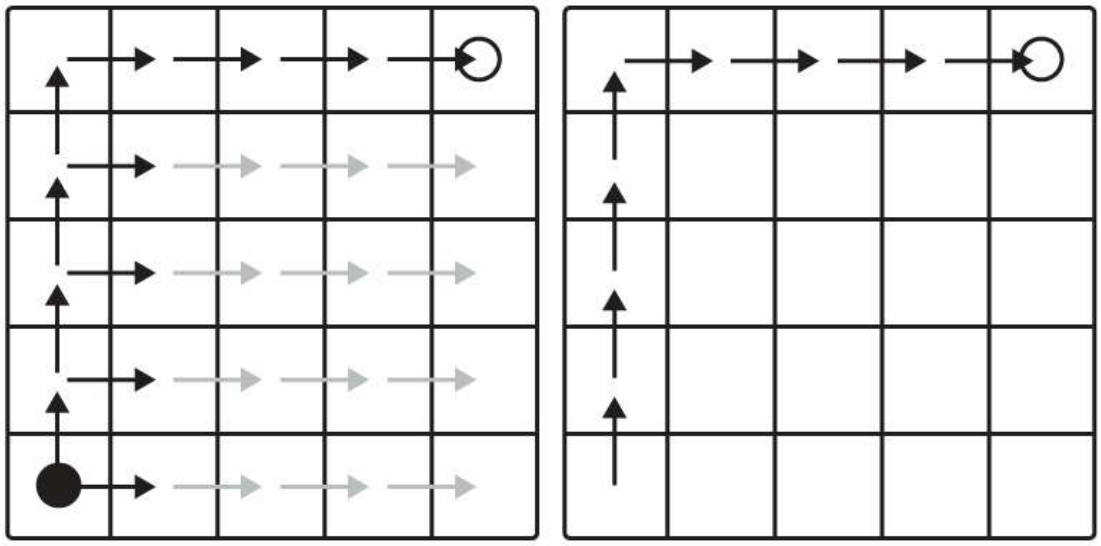
Horizontal-First

Horizontal: The original implementation specified by the JPS4 paper. This will search horizontally first (w/ vertical jumps). Faster with long vertical corridors, where vertical jumping will prune more nodes.



Vertical-First

Vertical: An alternate version of JPS4 that will search vertically first (w/ horizontal jumps). Faster with long horizontal corridors, where horizontal jumping will prune more nodes.



JPS4_Horizontal

Jump Point Search optimized for 4-connected grids. This offers performance gains over A* for most grids. **Horizontal-First** canonical ordering will favor moving horizontally before vertically when pruning nodes, but jump points are generated vertically. So this is more performant if you have long vertical corridors.

JPS4_Vertical

Jump Point Search optimized for 4-connected grids. This offers performance gains over A* for most grids. **Vertical-First** canonical ordering will favor moving vertically before horizontally when pruning nodes, but jump points are generated horizontally. So this is more performant if you have long horizontal corridors.

AStar4

Traditional A* pathfinding algorithm for 4-connected grids. This is a flexible, performant, and well-rounded approach widely used.

AStar8

Traditional A* pathfinding algorithm for 8-connected grids. This is a flexible, performant, and well-rounded approach widely used.

- Allows diagonal movement compared to JPS4
- ☑ `AllowCutCorners` will determine whether the navigation agent can slip through corners where two obstacles touch.

Editor Performance

It's important to note that Burst Jobs are much slower in the Unity Editor because of extra safety checks.

If you want to test performance or just want faster jobs in the editor, you can adjust these settings:

- Jobs > Burst > Safety Checks = `Off`
- Jobs > Burst > Native Debug Mode Compilation = `Off`
- Edit > Preferences > Jobs > Leak Detection Level = `Disabled`
- Edit > Preferences > Jobs > Enable Jobs Debugger = `Off`